

DIRUJITOS



Nombre	Apellido	Legajo	E-mail
Juan Ignacio	Cantarella	64509	jcantarella@itba.edu.ar
Máximo Daniel	Carranza	64538	mcarranza@itba.edu.ar
Alex	Köhler	64320	akohler@itba.edu.ar
Javier	Liu	64332	jaliu@itba.edu.ar

Tabla de contenidos

Introducción	2
Modelo computacional	3
Dominio	3
Características del lenguaje	3
Implementación	4
Frontend	5
Análisis Léxico (Flex)	5
Análisis Sintáctico (Bison)	5
Análisis Semántico (integrado)	5
Backend	5
Generación de código	5
Gestión de errores en runtime	6
Adicionales	6
Dificultades Encontradas	6
Futuras extensiones	7
Conclusiones	7
Bibliografía	8
Anexo - Guía de uso	9
Tipos de datos	9
Expresiones matemáticas	9
Expresiones booleanas	10
Sentencias	10
Funciones predefinidas	11
Funciones matemáticas	11
Funciones de dibujo	11
Constantes predefinidas	11
Comentarios	12
Ejemplos completos	13

Introducción

Con el surgimiento de nuevas tecnologías, la intersección entre las mismas y el arte aumenta cada vez más. Si bien existen ya herramientas para crear dibujos de manera programática, nos pareció interesante crear un Domain Specific Language que permite, de forma relativamente sencilla, crear imágenes que puedan ser visualizadas en la web.

Es por ello que surgió la idea de Dibujitos: mediante SVG, podemos gracias a su “resolución infinita” crear dibujos y fractales que sean escalables sin perder calidad. A su vez, los mismos tienen una alta compatibilidad (soporte con todos los navegadores web modernos) y son muy ligeros (como es XML, es muy liviano y fácil de comprimir).

Durante el desarrollo del proyecto se encontraron varias dificultades y se tomaron ciertas decisiones de diseño importantes. En este informe se explicará tanto el proceso de creación del lenguaje como los problemas y limitaciones que aparecieron a lo largo de su desarrollo.

Modelo computacional

Dominio

El proyecto consiste en el desarrollo de un Domain Specific Language (DSL) que permite crear dibujos 2D a partir de figuras geométricas básicas y expresiones matemáticas. El objetivo del lenguaje es proporcionar una sintaxis simple pero expresiva que permita a los usuarios definir diseños, patrones y figuras geométricas de manera programática.

Características principales del dominio:

- **Orientado a gráficos vectoriales:** Utiliza SVG como formato de salida
- **Matemáticamente expresivo:** Soporte para expresiones aritméticas y trigonométricas
- **Determinístico:** Todos los programas tienen ejecución finita y nada aleatorio.

La salida en formato SVG asegura:

- Compatibilidad con navegadores web modernos
- Escalabilidad sin pérdida de calidad
- Archivos ligeros y editables
- Integración con herramientas de diseño existentes

Características del lenguaje

Garantías de terminación:

- Los bucles **for** tienen iteración finita. El rango se determina antes del bloque y no mientras se ejecuta el bloque.
- No existen construcciones que puedan causar bucles infinitos.

Sistema de tipos:

- **Inmutabilidad de tipos:** Una vez asignado un tipo a una variable, no se puede cambiar. Habrá error de compilación semántico si se intenta asignar algún tipo de dato diferente al inicial.
- **Inferencia de tipos:** No es necesario declarar tipos explícitamente
- **Tipos primitivos:** **int**, **float**
- **Tipos compuestos:** Arrays homogéneos y vectores homogéneos

Gestión de memoria y alcance:

- **Scoping léxico:** Las variables tienen alcance de su bloque y padres
- **Gestión automática:** No se requiere gestión manual de memoria

Implementación

La arquitectura del compilador sigue el modelo clásico de tres fases, como se muestra en el siguiente diagrama conceptual:

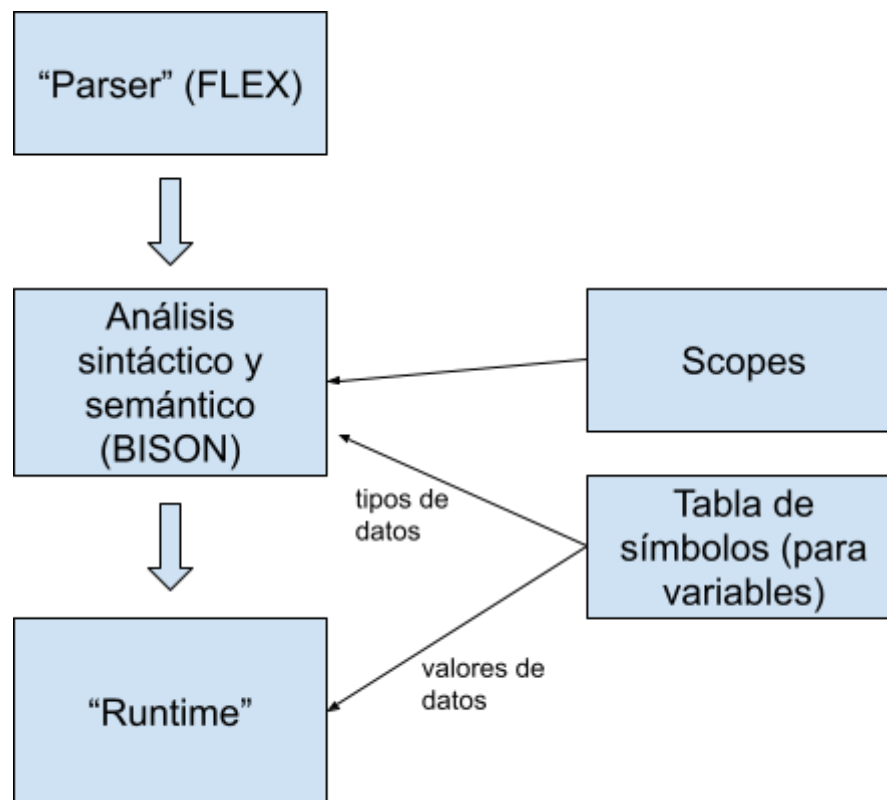


Figura 1: Diagrama de implementación

Tal cual como se ve en la *Figura 1*, el programa se divide en 3 lecturas:

- La lectura para el **"parsing"**. Genera "tokens", es decir, particiona el archivo de texto en substrings.
- El **análisis sintáctico y semántico**. Genera un árbol de tokens (Abstract Syntax Tree) y llena la tabla de símbolos. Informa sobre errores sintácticos y semánticos.
- La lectura final o **"runtime"**. Genera el SVG mientras evalúa los valores de las variables.

Frontend

Análisis Léxico (Flex)

- **Patrones de tokens:** Definidos mediante expresiones regulares
- **Contextos:** Manejo de strings con interpolación mediante contextos de Flex
- **Estados:**
 - **INITIAL:** Estado por defecto
 - **INTERP_STRING:** Dentro de strings con interpolación
 - **INTERP_EXPR:** Dentro de expresiones interpoladas
 - **MULTILINE_COMMENT:** Comentarios multilínea

Análisis Sintáctico (Bison)

- **Manejo de errores:** Recuperación de errores sintácticos con mensajes informativos

Análisis Semántico (integrado)

Decisión de diseño: El análisis semántico se realiza durante el análisis sintáctico mediante acciones semánticas en Bison.

Utiliza dos estructuras que están afuera del árbol sintáctico

Tabla de símbolos

- Nombres de variables
- Tipo de dato
- Valor (este no se escribe en esta fase, porque se actualiza en la fase siguiente durante la ejecución)
- Scope

Scope Stack

- Contiene todos scopes accesibles en cierto momento y cambia a medida de que se entre y salga de un scope

Verificaciones realizadas:

- Compatibilidad de tipos en operaciones
- Existencia de variables y funciones
- Validación de parámetros de funciones
- Verificación de índices de arrays
- Comprobación de scope de variables

Backend

Generación de código

El backend se encarga de:

- **Evaluación de expresiones:** Cálculo recursivo de valores de las variables en tiempo de ejecución
- **Generación de SVG:** Traducción directa a elementos SVG válidos
- **Manejo de contexto gráfico:** Estado de colores, capas y propiedades de dibujo

Estructura del SVG generado:

```
<svg width="800" height="600" xmlns="http://www.w3.org/2000/svg">
  <!-- Elementos ordenados por capa (layer) -->
  <circle cx="100" cy="100" r="50" fill="#ff0000" stroke="#000000"/>
  <!-- ... más elementos ... -->
</svg>
```

Gestión de errores en runtime

- **División por cero:** Detección y terminación controlada del programa
- **Acceso fuera de límites:** Validación de índices de arrays
- **Valores inválidos:** Verificación de parámetros de funciones matemáticas
- **Overflow:** Manejo de valores numéricos fuera de rango

Adicionales

Para facilitar el chequeo del correcto funcionamiento de nuestro compilador, creamos el ejecutable "tree_visualizer". La misma consiste en una herramienta de visualización de árboles que permite inspeccionar en detalle el proceso de compilación. La herramienta muestra el Árbol de Sintaxis Abstracta (AST), la tabla de símbolos (con tipos, valores y alcances), y la información de tipos inferidos para todas las variables y expresiones. Además, ayuda a identificar problemas en el análisis sintáctico y semántico, y se integra con la suite de pruebas. Se puede ejecutar manualmente con `./tree_visualizer <test_file.dib>`, o bien, al ejecutar `test.sh`, si se define la variable de entorno `SHOW_AST=true`, el árbol se imprime automáticamente en la consola. A su vez, el ejecutable `get_ast.sh` corre el tree visualizer para todos los tests en /accept (claramente, no tiene sentido ver el AST para los tests que fallan). Es una herramienta muy útil para comprender la estructura del código analizado, depurar problemas de análisis y validar la corrección del compilador.

Dificultades Encontradas

Cuando se empezó a realizar el análisis sintáctico, se pensó en implementar tipos de datos directamente en Bison, sin una tabla de símbolos. La idea era que existiesen tokens “integerExpression”, “vectorExpression”, entre otros. Esto fue posible de hacer con los booleanos (y luego, con arrays); pero cuando se trató de separar vectores de integers de floats, comenzaron a aparecer varios conflictos reduce/reduce. Al final se optó por un solo token “Expression” que utiliza funciones recursivas para averiguar el tipo de dato y el resultado de cada expresión.

Más tarde, se tomó la decisión de hacer chequeos semánticos en las mismas funciones de Bison (bisonActions.c). Los problemas comenzaron a aparecer con el “for”, pues requería lógica a ejecutar antes de entrar al árbol interno (mientras que Bison ejecuta las acciones después entrar al árbol por defecto). Esto se pudo resolver; sin embargo, el código de Bison quedó muy complejo y difícil de leer. Una alternativa más ordenada hubiese sido recorrer el árbol luego del análisis sintáctico (y antes del runtime) y realizar los chequeos semánticos allí.

A lo largo del trabajo hubo ciertos casos excepcionales que complicaron el desarrollo. Los arreglos, por ejemplo, se manejaban distinto al resto de tipos de datos. Al principio se guardaba cada elemento del arreglo en la tabla de símbolos en el análisis semántico, lo cual era erróneo, pues se asumía que el valor de estos elementos se mantenía constante durante todo el análisis. Algo parecido sucedía con el manejo de argumentos en funciones.

Futuras extensiones

Ahora mismo, una limitación importante es la falta de funciones definidas por el usuario. Si bien no son imprescindibles, esta adición facilitaría significativamente la tarea de realizar programas que dibujen fractales más complejos de forma recursiva o usen lógica que se repite a lo largo del programa. Su implementación implica reconsiderar la decisión de “todos los programas deben terminar”.

No hay coerción de tipos de dato. Ahora el lenguaje es bastante estricto en qué datos pasar. Por ejemplo la función `circle` exige que el radio sea un float, y si se le pasa un integer, entonces no compila lo cual es bastante molesto. Se deberá implementar conversión automática de integer a float y viceversa.

De la mano con la declaración de funciones `user`, viene la implementación de “import”. El mismo podría facilitar el desarrollo de ciertos programas y evitar la mala práctica de “copy-pastear” código repetido. En particular, para crear dibujos, el color podría ser función de la posición de cierto pixel, por lo que tendría sentido modularizar y crear funciones que luego puedan ser exportadas a otros archivos.

Por otro lado, con el fin de que nuestro lenguaje sea más declarativo y menos imperativo, podrían agregarse constructos o “features”. Por ejemplo, agrupación y composición de figuras geométricas. Así, podemos aplicar transformaciones/operaciones sobre los mismos (como rotar, trasladar o agrandar) sobre los grupos, facilitando el desarrollo de ciertos dibujos.

Conclusiones

En conclusión, con este proyecto hemos logrado comprender el funcionamiento fundamental de un compilador basado en Flex y Bison, así como el diseño completo de un lenguaje específico con su propia gramática y sintaxis. La elección de SVG como formato de salida resultó ser muy adecuada, ya que permitió generar gráficos vectoriales escalables de manera eficiente y precisa, además de brindar la oportunidad de explorar nuevas herramientas que serán de gran utilidad en futuros desarrollos.

Más allá de los aspectos puramente técnicos, este proyecto nos permitió experimentar con la intersección entre arte y tecnología, demostrando cómo un lenguaje formal puede ser utilizado para describir y generar imágenes visuales. De esta manera, se abre un espacio creativo que combina la lógica estructurada de la informática con la expresividad del arte visual.

Si bien la implementación actual tiene márgenes de mejora, el proyecto sienta una base sólida para futuras extensiones y optimizaciones.

Bibliografía

Bennett, J. P. (1996). *Introduction to compiling techniques: A first course using ANSI C, LEX and YACC* (2nd ed.). McGraw-Hill.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, & tools* (2nd ed.). Addison-Wesley.

Donnelly, C., & Stallman, R. M. (2003). *Bison: The Yacc-compatible parser generator* (8th ed.). GNU Press.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3a ed.). Pearson.

Anexo - Guía de uso

Para instrucciones de compilación y ejecución, referirse al README.

El lenguaje **Dibujitos** ofrece las siguientes construcciones y funcionalidades:

Importante: `x` e `y` son palabras reservadas. **NO USAR** para asignar variables.

Tipos de datos

Tipo de dato	Descripción	Sintaxis	Ejemplo
<code><id></code>	Identificador	Regex: <code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>a</code> , <code>id_of_array</code>
<code><float></code>	Número decimal	Regex: <code>`[+-]?([0-9]+.[0-9]*)`</code>	<code>(.[0-9]+))`</code>
<code><int></code>	Número entero	Regex: <code>[+-]?[0-9]+</code>	<code>5</code> , <code>-10</code> , <code>0x1A</code> (hexadecimal)
<code><vec></code>	Vector/punto 2D	<code>(<float>, <float>)</code>	<code>(5.0, 4.3)</code> , <code>(-1, 2)</code>
<code><arr></code>	Arreglo básico	<code>[<expr>, <expr>, ...]</code>	<code>[1, 2, 3, 4, 5]</code> , <code>[(1,1), (2,2)]</code>
<code><arr></code>	Arreglo por intervalo	<code>[<int>:<int>]</code>	<code>[1:5]</code> (equivale a <code>[1,2,3,4,5]</code>)
<code><interp_str></code>	String con interpolación	Entre comillas dobles, expresiones entre <code>{}</code>	<code>"Hola {nombre}, tu edad es {edad}"</code>

Expresiones matemáticas

Expresión	Descripción	Ejemplo
<code><num> <op> <num></code> donde <code><op></code> es <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Operaciones aritméticas básicas	<code>a = 1 + 1;</code>
<code><vec> <op> <vec></code> donde <code><op></code> es <code>+</code> , <code>-</code> , <code>*</code>	Operaciones vectoriales	<code>a = (1,1) + (2,2);</code>
<code>(<expr>)</code>	Paréntesis para precedencia	<code>a = (1 + 1) * 3;</code>
<code><arr>[<int>]</code>	Acceso a elemento de arreglo	<code>value = intArray[3];</code>

Expresión	Descripción	Ejemplo
<code><func_id>(<params>)</code>	Llamada a función	<code>a = sqrt(4.0);</code>
<code><vec>.x, <vec>.y</code>	Componentes de vector	<code>x_coord = point.x;</code>

Expresiones booleanas

Expresión	Descripción	Ejemplo
<code><expr> <op> <expr></code> donde <code><op></code> es <code>==, !=, <=, >=, <, ></code>	Operadores de comparación	<code>if (a == 1) { ... }</code>
<code><bool> and <bool>, <bool> or <bool></code>	Operadores lógicos binarios	<code>if (a >= 1 and a < 5) { ... }</code>
<code>not <bool></code>	Negación lógica	<code>if (not a == 1) { ... }</code>
<code>(<bool>)</code>	Paréntesis para precedencia	<code>if ((a > 0) and (b < 10)) { ... }</code>

Sentencias

Sentencia	Descripción	Ejemplo
<code><id> = <expr>;</code>	Asignación de variable	<code>a = 3; position = (100, 200);</code>
<code><id>[] = <arr>;</code>	Asignación de arreglo	<code>numbers[] = [1, 2, 3, 4];</code>
<code><id>[<int>] = <expr>;</code>	Asignación de elemento	<code>numbers[0] = 10;</code>
<code><id>.x = <expr>;, <id>.y = <expr>;</code>	Asignación de componente vectorial	<code>point.x = 5.0;</code>
<code>for <id> in <arr> { <stmts> }</code>	Bucle for-each	<code>for i in [1:9] { log "{i}"; }</code>
<code>if (<bool>) { <stmts> }</code>	Condicional	<code>if (x > 0) { log "positive"; }</code>
<code>if (<bool>) { <stmts> } else { <stmts> }</code>	Condicional con alternativa	<code>if (x > 0) { ... } else { ... }</code>
<code><func_id>(<params>);</code>	Llamada a función	<code>circle((100, 100), 50);</code>

Sentencia	Descripción	Ejemplo
<code>log <interp_str>;</code>	Salida a consola	<code>log "El valor es {variable}";</code>

Funciones predefinidas

Funciones matemáticas

Función	Descripción	Ejemplo
<code>sqrt(float)</code>	Raíz cuadrada	<code>sqrt(16.0) → 4.0</code>
<code>cos(float)</code> , <code>sin(float)</code> , <code>tan(float)</code>	Función trigonométricas	<code>cos(0.0) → 1.0</code>
<code>abs(float)</code>	Valor absoluto	<code>abs(-5.5) → 5.5</code>
<code>logf(float)</code>	Logaritmo natural	<code>logf(2.718) → ≈1.0</code>
<code>powf(float, float)</code>	Potencia	<code>powf(2.0, 3.0) → 8.0</code>
<code>roundf(float)</code>	Redondeo	<code>roundf(3.7) → 4.0</code>

Funciones de dibujo

Función	Descripción	Ejemplo
<code>stroke(int)</code>	Color de contorno	<code>stroke(RED);</code>
<code>fill(int)</code>	Color de relleno	<code>fill(0x00FF00);</code>
<code>layer(int)</code>	Orden Z (0=atrás, 1=adelante)	<code>layer(FRONT);</code>
<code>circle(vec, float)</code>	Dibuja círculo	<code>circle((0,0), 50.0);</code>
<code>line(vec, vec)</code>	Dibuja línea	<code>line((0,0), (100,100));</code>
<code>curve(vec, ...)</code>	Curva de Bézier	<code>curve((0,0), (50,100), (100,0));</code>
<code>polygon(vec, ...)</code>	Polígono	<code>polygon((0,0), (50,0), (25,50));</code>

Constantes predefinidas

```
// Colores
RED, GREEN, BLUE, YELLOW, BLACK, ORANGE, INDIGO, PURPLE

// Capas
BACK = 0, FRONT = 1
```

Comentarios

Los comentarios de línea se realizan con `//` y solo afectan el texto a la derecha:

Los comentarios multilínea se delimitan con `/*` y `*/`:

```
// Este es un comentario completo  
  
x = 5; // Comentario al final de línea
```

```
/*  
  
Este es un comentario  
  
de múltiples líneas  
  
*/
```

Ejemplos completos

Dibujo de espiral con gradiente.

```
circleRadius = 4.;           // radius of each small circle
steps = 100;                 // number of circles per arm
armOffset = 3.14159;         // pi radians offset between arms
separation = 20;

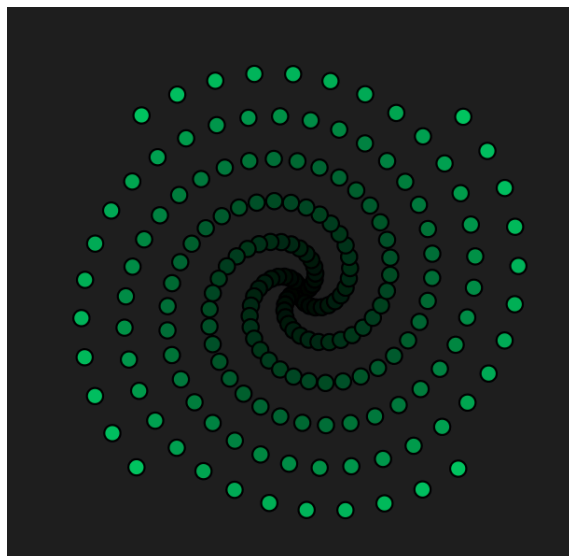
// Draw spiral arms
for i in [0:steps - 1] {
    t = i * 0.24;
    r = t * 5.;
    hue = i * 3.;

    // First arm
    angle1 = t * separation;
    x1 = cos(angle1) * r;
    y1 = sin(angle1) * r;
    fill(1 * i + 256 * 2 * i + 256 * 256 * i);

    circle((x1, y1), circleRadius);

    // Second arm (offset by pi)
    angle2 = t * separation + armOffset;
    x2 = cos(angle2) * r;
    y2 = sin(angle2) * r;

    circle((x2, y2), circleRadius);
}
```



Dibujo de una casa simple

```
// Dibujo de una casa simple
fill(YELLOW);
stroke(BLACK);

// Base de la casa
polygon((0,0), (200,0), (200,100), (0,100));

// Techo
fill(RED);
polygon((0,100), (200,100), (100,150));

// Puerta
fill(BLACK);
polygon((80,0), (80,70), (120,70), (120,0));

// Ventana
fill(BLUE);
circle((150,50), 15.);

log "Casa dibujada exitosamente";
```

